

Rishad Hossain

Section: Mon/Wed 4-5:15pm

Question 1:

Steps heuristic:

A. Pseudocode:

[

distance = 0

for each position i:

 if state[i] != goal[i]:

 distance += 1

return distance

]

B. This heuristic counts the number of letter positions that still need to be corrected. Since each legal action can change only one letter, then each incorrect position requires one action which is the unavoidable remaining cost it lower bounds.

C. Since every action has a cost of 1, distance in the pseudocode would eventually represent the count of every incorrect position and would therefore mean at least distance actions are required. Which would mean that the heuristic cannot overestimate the true optimal remaining cost and is admissible. At the goal, there are no differing positions, so the heuristic returns 0.

Scrabble heuristic:

A. Pseudocode:

```
[  
total = 0  
for each position i:  
    if state[i] != goal[i]:  
        total += scrabble_value(goal[i])  
return total  
]
```

B. This heuristic lower-bounds the total cost of each letter that was used to change the incorrect positions to achieve the goal state.

C. The cost of each action is the value of a newly inserted letter. Which means every incorrect position would need to at least pay the cost for each letter that is required to reach the goal state. Any other moves would only increase our cost. So, because our heuristic only counts the cost for what is needed, it does not overestimate the optimal remaining cost. At the goal, there are no incorrect positions, so the heuristic returns 0.

Frequency heuristic:

A. Pseudocode:

```
[  
distance = 0  
for each position i in the word:  
    if state[i] != goal[i]:  
        distance = distance + 1  
if distance == 0:  
    return 0  
return distance + rarity(goal)  
]
```

B. This heuristic lower-bounds at least distance actions because the amount of incorrect positions need to be changed to eventually reach the goal state. And when it reaches the goal, it would need to pay the rarity cost

C. The frequency cost of $1 + \text{rarity}(w)$ tells us that every action needed to correct a distance amount of positions would cost at least 1. The path to the goal would eventually need to pay the rarity cost, the 1 would already be included with the distance cost. So, distance plus $\text{rarity}(w)$ would be the lower bound and does not overestimate the optimal remaining cost. At the goal, the heuristic returns 0.

Question 2:

Word Pair	Cost Measure	UCS Cost	UCS Expanded	A* Cost	A* Expanded	Reduction
WHY → ASK	Steps	5	773	5	16	97.93%
WHY → ASK	Scrabble	9	765	9	13	98.30%
WHY → ASK	Frequency	38.8554	808	38.8554	634	21.53%
BEAR → DUCK	Steps	4	691	4	6	99.13%
BEAR → DUCK	Scrabble	11	2981	11	6	99.80%
BEAR → DUCK	Frequency	38.2402	1084	38.2402	302	72.14%

1. Yes, for all six experiments the UCS and A* costed the same.
2. Under both word pairs, Scrabble had the highest reduction percentage.
3. For both word pairs, frequency was the least informative as it had the lowest reduction percentage.
4. The steps heuristic counts how many incorrect positions, which is useful information for A* about how many actions are required. The scrabble heuristic takes it step further as it considers both incorrect positions and the values of the letters that must be inserted eventually, which gave A* a more effective search. The frequency heuristic, while admissible, does not consider the rarity costs for the intermediate words that might be entered along the path to the goal state. This could cause many states to have similar heuristic estimates even if their true costs are different. This can cause A* to expand more nodes because the frequency heuristic is not as informative.

Question 3:

Steps: Yes, A legal move in either direction would only change exactly one letter and have the same cost. Changing a distance amount of letter positions to go from W1->W2 would be the same as W2->W1. So, the optimal costs would be the same.

Scrabble: No, while it is reversible, changing letters have different costs. So the costs for a letter to change in one direction may cost differently in the reverse direction. An example would be

dog to cog and cog to dog. When d->c, c has a cost of 3 so it would incur a cost of 3. However, c->d would incur a cost of 2 because of the cost of the letter d. This shows that the optimal costs can be different.

Frequency: No, the cost is also direction dependent like scrabble because words like dog and cog could have different rarity values which would change the optimal costs. For proof, after a test, dog has a rarity: 5.819885; and cog has a rarity: 11.393166.

Question 4:

1. OpenAI ChatGPT — GPT-5.6 Sol

Using the provided prompt given, it provided this heuristic:

$$h(n) = \begin{cases} 0 & \text{if } n \text{ is the goal} \\ d(n, g) + \text{rarity}(g) & \text{otherwise} \end{cases}$$

where $d(n, g)$ is the number of positions in which the current word differs from the goal.

2. Pseudocode:

distance = 0

for each position i:

 if state[i] != goal[i]:

 distance = distance + 1

if distance == 0:

 return 0

return distance + rarity(goal)

3. Yes, this heuristic is admissible because it only counts the unavoidable costs for a frequency heuristic which is the minimum number of required letter changes and the rarity cost of the final goal.

4. No repair was required as it was admissible.

5. The proposed heuristic from the AI for each word pair, had the same solution costs; however, it expanded a different amount of nodes. H0 from our question 2 tests expanded 634 nodes for Why->Ask and 302 nodes for Bear->Duck. However, the proposed heuristic expanded 755 nodes for Why->Ask and 720 nodes for Bear->Duck. So, while the optimal costs stayed the same, the additional expanded nodes does not support the need to use the proposed heuristic.